ระบบประเมินผลบุคลากร (ส่วน Backend)

Technology Stack
- NodeJS, JavaScript
- Prisma
- MySQL Database
- Swagger
- Other necessary packages

1️⃣ Domain & Data Model (แกนของ Backend)

ความสัมพันธ์ที่ถูกต้อง

User (ADMIN | EVALUATOR | EVALUATEE)
Department

Evaluation
└─ Topic
  └─ Indicator (type, weight, requireEvidence)

ตัวชี้วัดทุกตัวในแบบประเมินรวมกันต้องได้ 100% หากเกินจะไม่อนุญาตให้แก้/เพิ่มตัวชี้วัด

Assignment
(evaluationId, evaluatorId, evaluateeId)

Evidence
(evaluatee -> indicator)

EvaluationResult
(evaluator -> indicator -> score)

🔑 **สำคัญมาก**

- Evidence = ของ Evaluatee

- Score = ของ Evaluator
- ห้ามรวมสองอย่างนี้ไว้ตารางเดียว

Prisma Schema (ฉบับ backend จริง)

```
model User {
  id            Int      @id @default(autoincrement())
  name          String
  email         String   @unique
  passwordHash  String
  role          Role
  departmentId  Int?
  department    Department? @relation(fields: [departmentId], references: [id])
  createdAt     DateTime @default(now())
}

model Department {
  id    Int    @id @default(autoincrement())
  name  String
}

model Evaluation {
  id        Int      @id @default(autoincrement())
  name      String
  startAt   DateTime
  endAt     DateTime
  status    EvaluationStatus
  topics    Topic[]
}

model Topic {
  id           Int      @id @default(autoincrement())
  evaluationId Int
  name         String
  evaluation   Evaluation @relation(fields: [evaluationId], references: [id])
  indicators   Indicator[]
}

model Indicator {
  id             Int      @id @default(autoincrement())
  topicId        Int
  name           String
```

```
  type           IndicatorType
  weight         Float
  requireEvidence Boolean
  topic          Topic @relation(fields: [topicId], references: [id])
}

model Assignment {
  id            Int @id @default(autoincrement())
  evaluationId  Int
  evaluatorId   Int
  evaluateeId   Int

  evaluation Evaluation @relation(fields: [evaluationId], references: [id])
  evaluator  User       @relation("Evaluator", fields: [evaluatorId], references: [id])
  evaluatee  User       @relation("Evaluatee", fields: [evaluateeId], references: [id])

  @@unique([evaluationId, evaluatorId, evaluateeId])
}

model Evidence {
  id           Int @id @default(autoincrement())
  indicatorId  Int
  evaluateeId  Int
  filePath     String
  mimeType     String
  sizeBytes    Int
  createdAt    DateTime @default(now())

  @@unique([indicatorId, evaluateeId])
}

model EvaluationResult {
  id            Int @id @default(autoincrement())
  assignmentId  Int
  indicatorId   Int
  score         Int?   // 1–4 or 0/1
  createdAt     DateTime @default(now())
}

enum Role {
  ADMIN
  EVALUATOR
  EVALUATEE
}
```

```
enum IndicatorType {
  SCALE_1_4
  YES_NO
}

enum EvaluationStatus {
  DRAFT
  OPEN
  CLOSED
}
```

2️⃣ REST API Design (Backend Contract จริง)

Authentication

POST /api/auth/login
POST /api/auth/register
GET /api/auth/me

Admin APIs

Evaluation

POST /api/admin/evaluations
GET /api/admin/evaluations
PATCH /api/admin/evaluations/:id
DELETE /api/admin/evaluations/:id

Topic / Indicator

POST /api/admin/evaluations/:id/topics
POST /api/admin/topics/:id/indicators
PATCH /api/admin/topics/:id
DELETE /api/admin/topics/:id

PATCH /api/admin/indicators/:id
DELETE /api/admin/indicators/:id

Assignment

POST /api/admin/assignments

> ❗ Backend ต้อง enforce:

- evaluatorId ≠ evaluateeId
- assignment ซ้ำ → 409 DUPLICATE_ASSIGNMENT

Evaluatee APIs (แนบหลักฐาน)

POST /api/me/evaluations/:evaluationId/evidence

Validation:

- requireEvidence = true → ต้องแนบไฟล์
- 1 indicator = 1 file
- exe → 415
- 10MB → 413

Evaluator APIs (ให้คะแนน)

GET /api/evaluator/evaluations/:evaluationId
POST /api/evaluator/assignments/:id/score

Rules:

- ให้คะแนนได้เฉพาะ assignment ของตัวเอง
- ถ้า requireEvidence=true และยังไม่มี evidence → 400

Reports

ผลคะแนนรวม (IDOR)

GET /api/reports/evaluation/:evaluationId/result

RBAC:

- ADMIN → ทุกคน
- EVALUATOR → เฉพาะ evaluatee ที่ตนประเมิน
- EVALUATEE → เฉพาะของตนเอง

3️⃣ Business Logic ที่ “ต้องอยู่ Backend เท่านั้น”

สูตรคะแนน (Single Source of Truth)

```
if (indicator.type === 'SCALE_1_4') {
  adjusted = (score / 4) * indicator.weight
}

if (indicator.type === 'YES_NO') {
  adjusted = (score === 1 ? 1 : 0) * indicator.weight
}
```

❌ Frontend ห้ามคิดคะแนนเอง

✅ Backend คำนวณและส่งผลลัพธ์มาเท่านั้น

Evaluation Completion

Evaluation ถือว่า "เสร็จสมบูรณ์" เมื่อ
- Evaluator ให้คะแนนครบทุก Indicator

ก่อนหน้านั้น:

- Evaluatee ห้ามดูผล
- API ต้องตอบ 403 หรือ 409 ตามกรณี

ข้อกำหนดเพิ่มเติม

- Seed ข้อมูลตัวอย่าง 1 admin, 1 evaluator, 1 evaluatee, 1 evaluation, 2 topics (with 2 indicators each)
- สร้างไฟล์ postman collection เพื่อใช้ยิง api ผ่าน postman
- ทำไฟล์ Readme เพื่ออธิบายวิธีการเรียก API และบัญชีผู้ใช้ที่สร้างด้วยเพื่อให้ง่ายต่อการทดสอบ login
-